Exercise 3.1: ArrayList Shuffling

```
import java.util.ArrayList;

public class Exercise3_1 {
    public static void shuffle(ArrayList<Number> list) {
        for (int i = list.size() - 1; i > 0; i--) {
            int j = (int) (Math.random() * (i + 1));

            Number temp = list.get(i);
            list.set(i, list.get(j));
            list.set(j, temp);
        }
    }

    public static void main(String[] args) {
        ArrayList<Number> list = new ArrayList<>();

        list.add(10);
        list.add(3.14);
        list.add(25);
        list.add(8.5);
        list.add(42);

        System.out.println("Before shuffling: " + list);
        shuffle(list);
        System.out.println("After shuffling:  " + list);
    }
}
```

Exercise 3.2: `Comparable` Interface

ComparableCircle.java:

```
public class ComparableCircle extends Circle implements
Comparable<ComparableCircle> {

    public ComparableCircle() {
        super();
    }

    public ComparableCircle(double radius) {
        super(radius);
    }

    @Override
    public int compareTo(ComparableCircle other) {
        double thisRadius = getRadius();
        double otherRadius = other.getRadius();

        if (thisRadius > otherRadius) {
            return 1;
        } else if (thisRadius < otherRadius) {
            return -1;
        } else {
            return 0;
        }
    }

    @Override
    public String toString() {
        return "ComparableCircle[radius=" + getRadius() + ", area=" +
           getArea() + "]";
    }
}
```

TestComparableCircle.java:

```
public class TestComparableCircle {
    public static ComparableCircle max(ComparableCircle c1,
      ComparableCircle c2) {
        if (c1.compareTo(c2) >= 0) {
            return c1;
        } else {
            return c2;
        }
    }
    public static void main(String[] args) {
        ComparableCircle circle1 = new ComparableCircle(3.0);
        ComparableCircle circle2 = new ComparableCircle(5.0);

        System.out.println("Circle 1: " + circle1);
        System.out.println("Circle 2: " + circle2);

        ComparableCircle larger = Max.max(circle1, circle2);
        System.out.println("The larger circle is: " + larger);
    }
}
```

Exercise 3.3: Deep Copy

MyStack.java:

```
import java.lang.reflect.Method;
import java.util.ArrayList;

public class MyStack implements Cloneable {
    private ArrayList<Object> list = new ArrayList<>();

    public void push(Object o) {
        list.add(o);
    }

    public Object pop() {
        Object o = list.get(getSize() - 1);
        list.remove(getSize() - 1);
        return o;
    }

    public Object peek() {
        return list.get(getSize() - 1);
    }

    public int search(Object o) {
        return list.lastIndexOf(o);
    }

    public boolean isEmpty() {
        return list.isEmpty();
    }

    public int getSize() {
        return list.size();
    }

    @Override
    public String toString() {
        return "stack: " + list.toString();
    }

    @Override
    public Object clone() {
        try {
            MyStack copiedStack = (MyStack) super.clone();
            copiedStack.list = new ArrayList<>();

            for (Object item : this.list) {
                copiedStack.list.add(cloneElement(item));
            }

            return copiedStack;
        } catch (CloneNotSupportedException e) {
            return null;
        }
    }

    private Object cloneElement(Object item) {
        if (item == null) {
```

```
            return null;
        }

        if (item instanceof Cloneable) {
            try {
                Method method = item.getClass().getMethod("clone");
                return method.invoke(item);
            } catch (Exception e) {
                return item;
            }
        }

        return item;
    }
}
```

TestMyStack.java:

```
class TestMyStack {
    public static void main(String[] args) {
        MyStack stack1 = new MyStack();
        stack1.push("A");
        stack1.push("B");
        stack1.push(new StringBuilder("C"));

        MyStack stack2 = (MyStack) stack1.clone();

        System.out.println("Original: " + stack1);
        System.out.println("Clone:    " + stack2);
        System.out.println("Same object? " + (stack1 == stack2));
    }
}
```

Exercise 3.4: Abstract Superclass for Employees

Employee.java:

```
public abstract class Employee {
    private String name;
    private String id;

    public Employee(String name, String id) {
        this.name = name;
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public String getId() {
        return id;
    }

    public abstract double calculateSalary();

    @Override
    public String toString() {
        return "Employee[name=" + name + ", id=" + id + "]";
    }
}
```

FullTimeEmployee.java:

```
public class FullTimeEmployee extends Employee {
    private double monthlySalary;

    public FullTimeEmployee(String name, String id, double monthlySalary) {
        super(name, id);
        this.monthlySalary = monthlySalary;
    }

    @Override
    public double calculateSalary() {
        return monthlySalary;
    }

    @Override
    public String toString() {
        return "FullTimeEmployee[name=" + getName() +
               ", id=" + getId() +
               ", monthlySalary=" + monthlySalary + "]";
    }
}
```

PartTimeEmployee.java:

```
public class PartTimeEmployee extends Employee {
    private double hoursWorked;
    private double hourlyRate;
```

```
    public PartTimeEmployee(String name,
                String id,
                double hoursWorked,
                double hourlyRate)
    {
        super(name, id);
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    @Override
    public double calculateSalary() {
        return hoursWorked * hourlyRate;
    }

    @Override
    public String toString() {
        return "PartTimeEmployee[name=" + getName() +
               ", id=" + getId() +
               ", hoursWorked=" + hoursWorked +
               ", hourlyRate=" + hourlyRate + "]";
    }
}
```

TestEmployee.java:

```
public class TestEmployee {
    public static void main(String[] args) {
        Employee[] employees = new Employee[2];
        employees[0] = new FullTimeEmployee("Alice", "FT001", 8000);
        employees[1] = new PartTimeEmployee("Bob", "PT001", 60, 50);

        for (Employee employee : employees) {
            System.out.println(employee);
            System.out.println("Salary: " + employee.calculateSalary());
            System.out.println();
        }
    }
}
```

Exercise 3.5: Interface for Discountable Products

Discountable.java:

```
public interface Discountable {
    double getDiscountedPrice(double percent);
}
```

Product.java:

```
public class Product {
    private String name;
    private double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }
}
```

Book.java:

```
public class Book extends Product implements Discountable {
    public Book(String name, double price) {
        super(name, price);
    }
    @Override
    public double getDiscountedPrice(double percent) {
        return getPrice() * (1 - percent / 100.0);
    }
    @Override
    public String toString() {
        return "Book[name=" + getName() + ", originalPrice=" +
          getPrice() + "]";
    }
}
```

TestBook.java:

```
public class TestBook {
    public static void main(String[] args) {
        Book book = new Book("Java Programming", 120.0);
        double percent = 20.0;
        System.out.println(book);
        System.out.println("Original price: " + book.getPrice());
        System.out.println("Discounted price: " +
          book.getDiscountedPrice(percent));
    }
}
```

Exercise 3.6: Abstract Class vs. Interface

1. This code will **not** compile successfully.

   Reason: In an interface, all variables are implicitly `public static final`. Therefore `name` must be initialized when declared. Writing `String name;` is invalid because it is effectively a constant without an assigned value.

   There are two easy valid fixes:
   1) Change `Animal` to an `abstract class`.
   2) Initialize `name` with a `value`.

2. This code will **not** compile successfully.

   Reason: A normal interface method cannot have a method body. To fix this, remove the method body.

3. An abstract class cannot be instantiated using `new`. Abstract classes may have constructors, but those constructors are only called through subclasses.

   How to solve it: Create a concrete subclass of `MyClass` and instantiate that subclass; or remove the instantiation of `MyClass` using `new`.

Exercise 3.7: Comparable Interface for Students

Student.java:

```
public class Student implements Comparable<Student> {
    private String name;
    private double mark;

    public Student(String name, double mark) {
        this.name = name;
        this.mark = mark;
    }

    public String getName() {
        return name;
    }

    public double getMark() {
        return mark;
    }

    @Override
    public int compareTo(Student other) {
        if (this.mark > other.mark) {
            return 1;
        } else if (this.mark < other.mark) {
            return -1;
        } else {
            return this.name.compareTo(other.name);
        }
    }

    @Override
    public String toString() {
        return "Student[name=" + name + ", mark=" + mark + "]";
    }
}
```

TestStudent.java:

```
import java.util.Arrays;

public class TestStudent {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Charlie", 82.5),
            new Student("Alice", 90.0),
            new Student("Bob", 90.0),
            new Student("David", 75.0)
        };

        Arrays.sort(students);

        for (Student student : students) {
            System.out.println(student);
        }
    }
}
```

Exercise 3.8: Using the Number Class

```
public class Exercise3_8 {
    public static void main(String[] args) {
        Number[] numbers = {
            Integer.valueOf(10),
            Double.valueOf(2.5),
            7,
            8.75,
            Integer.valueOf(20)
        };

        double sum = 0.0;

        for (Number number : numbers) {
            System.out.println("Value: " + number + ", Type: " +
               number.getClass().getSimpleName());
            sum += number.doubleValue();
        }

        System.out.println("Final sum: " + sum);
    }
}
```